Name: Muhammad Noor
Roll No: i232520
Section: DS–A
Submission Date: *November 23, 2025*
Assignment: 3

Abstract

This report compares four classical shortest-path algorithms: **Dijkstra**, **Bellman–Ford**, **Floyd–Warshall**, and **Johnson**. It describes the objectives and approach, presents clear pseudocode for each algorithm, shows sample input/output, and provides experimental analysis (runtime, memory, relaxations) across three graph families (Sparse, Dense, Mixed). The experiments were implemented and exported as CSV. The study highlights algorithm suitability per graph type and discusses handling of negative weights and negative cycles.

Contents

1 Objective and Approach **2**
1.1 Objectives . . . 2
1.2 Experimental Approach (Implementation Notes) . . . 2

2 Algorithm Pseudocode **2**
2.1 Dijkstra (Single-Source, Non-Negative Weights) . . . 2
2.2 Bellman–Ford (Allows Negative Weights) . . . 2
2.3 Floyd–Warshall (All-Pairs) . . . 3
2.4 Johnson (All-Pairs with Reweighting) . . . 3

3 Sample Input / Output (Screenshots) **3**

4 Experimental Analysis and Comparison Table **5**
4.1 Metrics Captured . . . 5
4.2 Comparison Table . . . 6

5 Answers to the Assignment Questions **6**
5.1 Key Differences Between Single-Source and All-Pairs Shortest-Path Algorithms . 6
5.2 Why Dijkstra Fails with Negative Edge Weights . . . 6
5.3 How Johnson Combines Bellman–Ford and Dijkstra . . . 6
5.4 Complexity Analysis of Algorithms . . . 7
5.5 Algorithm Suitability Per Graph Type . . . 7

6 Conclusion **7**

List of Figures

1 Testing on Sample Input . . . 3
2 Testing on Matrix Input . . . 4
3 Testing on Sparse Matrix . . . 4
4 Testing on Dense Matrix . . . 5
5 Testing on Mixed Matrix . . . 5

1 Objective and Approach

1.1 Objectives

- Implement and evaluate Dijkstra, Bellman–Ford, Floyd–Warshall, and Johnson algorithms.
- Measure runtime (ms), number of relaxations, memory usage (peak), and detect negative cycles.
- Compare performance across three graph families: **Sparse**, **Dense**, and **Mixed**.
- Export experimental results to CSV for tabulation and plotting.

1.2 Experimental Approach (Implementation Notes)

Experiments were automated by the supplied Python script which:

- Generates graph families (sparse, dense, mixed), optionally directed or undirected.
- Runs single-source algorithms (Dijkstra/Bellman–Ford) and all-pairs algorithms (Floyd–Warshall/Johnson).
- Measures runtime using high-resolution timers and peak memory via `tracemalloc`.
- Exports CSV rows: Algorithm, Graph Type, Time (ms), Supports Negative Weights, Complexity, Relaxations, Memory (KB).

2 Algorithm Pseudocode

2.1 Dijkstra (Single-Source, Non-Negative Weights)

```
DIJKSTRA(G, source):
for each v in G: dist[v] <- +inf; parent[v] <- -1
dist[source] <- 0
insert (0, source) into min-heap
while heap not empty:
    (d,u) <- heap.pop()
    if d > dist[u]: continue
    for (v, w) in neighbors(u):
        if dist[u] + w < dist[v]:
            dist[v] <- dist[u] + w
            parent[v] <- u
            heap.push((dist[v], v))
return dist, parent
```

2.2 Bellman–Ford (Allows Negative Weights)

```
BELLMAN_FORD(G, source):
for each v: dist[v] <- +inf; parent[v] <- -1
dist[source] <- 0
for i in 1 .. |V|-1:
    updated <- false
    for each edge (u,v,w):
        if dist[u] + w < dist[v]:
            dist[v] <- dist[u] + w
            parent[v] <- u
            updated <- true
    if not updated: break
for each edge (u,v,w):
```

```
    if dist[u] + w < dist[v]: negative_cycle <- true
return dist, parent, negative_cycle
```

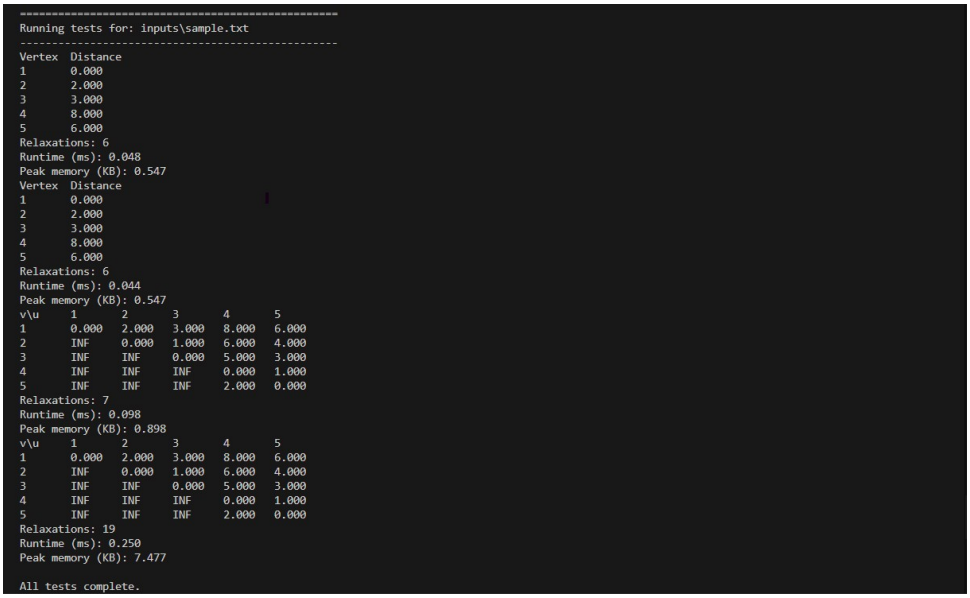
2.3 Floyd–Warshall (All-Pairs)

```
FLOYD_WARSHALL(G):
let n = |V|
dist[i][j] <- +inf
dist[v][v] <- 0
for each edge (u,v,w): dist[u][v] <- w
for k = 1..n:
    for i = 1..n:
        for j = 1..n:
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] <- dist[i][k] + dist[k][j]
negative_cycle <- any(dist[i][i] < 0)
```

2.4 Johnson (All-Pairs with Reweighting)

```
JOHNSON(G):
add new node s with edges (s -> v) weight 0
run Bellman-Ford from s to compute h[v]
reweight w'(u,v) = w(u,v) + h[u] - h[v]
for each vertex u:
    run Dijkstra from u on reweighted graph
    convert result back: d[u][v] = d'[u][v] - h[u] + h[v]
```

3 Sample Input / Output (Screenshots)



```
==============================================
Running tests for: inputs\sample.txt
----------------------------------------------
Vertex  Distance
1       0.000
2       2.000
3       3.000
4       8.000
5       6.000
Relaxations: 6
Runtime (ms): 0.048
Peak memory (KB): 0.547
Vertex  Distance
1       0.000
2       2.000
3       3.000
4       8.000
5       6.000
Relaxations: 6
Runtime (ms): 0.044
Peak memory (KB): 0.547
v\u     1       2       3       4       5
1       0.000   2.000   3.000   8.000   6.000
2       INF     0.000   1.000   6.000   4.000
3       INF     INF     0.000   5.000   3.000
4       INF     INF     INF     0.000   1.000
5       INF     INF     INF     2.000   0.000
Relaxations: 7
Runtime (ms): 0.098
Peak memory (KB): 0.898
v\u     1       2       3       4       5
1       0.000   2.000   3.000   8.000   6.000
2       INF     0.000   1.000   6.000   4.000
3       INF     INF     0.000   5.000   3.000
4       INF     INF     INF     0.000   1.000
5       INF     INF     INF     2.000   0.000
Relaxations: 19
Runtime (ms): 0.250
Peak memory (KB): 7.477

All tests complete.
```

Figure 1: Testing on Sample Input

```
==================================================
Running tests for: inputs\block_matrix.txt
==================================================
Vertex  Distance
1       0.000
2       5.000
3       7.000
Relaxations: 2
Runtime (ms): 0.044
Peak memory (KB): 0.398
Vertex  Distance
1       0.000
2       5.000
3       7.000
Relaxations: 2
Runtime (ms): 0.037
Peak memory (KB): 0.398
v\u     1       2       3
1       0.000   5.000   7.000
2       INF     0.000   2.000
3       INF     INF     0.000
Relaxations: 1
Runtime (ms): 0.066
Peak memory (KB): 0.555
v\u     1       2       3
1       0.000   5.000   7.000
2       INF     0.000   2.000
3       INF     INF     0.000
Relaxations: 6
Runtime (ms): 0.237
Peak memory (KB): 4.102
```

Figure 2: Testing on Matrix Input

	A	B	C	D	E	F	G
1	Algorithm	Graph Typ	Time (ms)	Works wit	Complexit	Number o	Memory (K
2	Dijkstra	Sparse	0.0661	No	O(m + n lo	9	0.7188
3	Bellman-F	Sparse	0.0659	Yes	O(nm)	9	0.7578
4	Floyd-War	Sparse	0.4445	Yes	O(n^3)	85	2.625
5	Johnson	Sparse	0.8469	Yes	O(nm + n^	91	13.8203
6	Dijkstra	Sparse	0.069	No	O(m + n lo	31	0.6406
7	Bellman-F	Sparse	0.0635	Yes	O(nm)	32	0.6328
8	Floyd-War	Sparse	5.5961	Yes	O(n^3)	1141	23.2812
9	Johnson	Sparse	3.8151	Yes	O(nm + n^	861	52.9531
10	Dijkstra	Sparse	0.1072	No	O(m + n lo	57	1.0156
11	Bellman-F	Sparse	0.1036	Yes	O(nm)	92	0.875
12	Floyd-War	Sparse	30.935	Yes	O(n^3)	4549	73.3203
13	Johnson	Sparse	12.7996	Yes	O(nm + n^	2647	121.5781

Figure 3: Testing on Sparse Matrix

A1 | Algorithm

	A	B	C	D	E	F	G
1	Algorithm	Graph Typ	Time (ms)	Works wit	Complexit	Number o	Memory (KB)
2	Dijkstra	Dense	4.5792	No	O(m + n lo	262	9.6016
3	Bellman-F	Dense	10.503	Yes	O(nm)	698	2.0859
4	Floyd-War	Dense	586.4255	Yes	O(n^3)	32572	268.7812
5	Johnson	Dense	427.1577	Yes	O(nm + n^	26344	2792.867
6	Dijkstra	Dense	8.7678	No	O(m + n lo	443	13.7031
7	Bellman-F	Dense	23.9497	Yes	O(nm)	1298	3.8203
8	Floyd-War	Dense	2233.337	Yes	O(n^3)	81351	609.7656
9	Johnson	Dense	1428.123	Yes	O(nm + n^	63014	6392.594
10	Dijkstra	Dense	15.6183	No	O(m + n lo	589	19.3359
11	Bellman-F	Dense	59.3013	Yes	O(nm)	1739	5.7734
12	Floyd-War	Dense	4439.946	Yes	O(n^3)	157623	1094.25
13	Johnson	Dense	3319.233	Yes	O(nm + n^	115516	11370
14							

Figure 4: Testing on Dense Matrix

A1 | Algorithm

	A	B	C	D	E	F	G
1	Algorithm	Graph Typ	Time (ms)	Works wit	Complexit	Number o	Memory (KB)
2	Bellman-F	Mixed	8.1234	Yes	O(nm)	2926	1.3828
3	Floyd-War	Mixed	30.4713	Yes	O(n^3)	13689	21.7578
4	Johnson	Mixed	4.5293	Yes	O(nm + n^	2975	39.5859
5	Bellman-F	Mixed	53.8617	Yes	O(nm)	36845	1.2422
6	Floyd-War	Mixed	346.7957	Yes	O(n^3)	203059	113.5938
7	Johnson	Mixed	56.4951	Yes	O(nm + n^	37007	193.6641

Figure 5: Testing on Mixed Matrix

4 Experimental Analysis and Comparison Table

4.1 Metrics Captured

- Runtime (ms)
- Number of relaxations
- Peak memory (KB)
- Negative cycle detection

4.2 Comparison Table

Algorithm	Graph Type	Time (ms)	Neg Weights	Complexity	Relaxations	Memory (KB)
Dijkstra	Sparse	0.0661	No	$O(m + n \log n)$	9	0.7188
Bellman–Ford	Sparse	0.0659	Yes	$O(nm)$	9	0.7578
Floyd–Warshall	Sparse	0.4445	Yes	$O(n^3)$	85	2.625
Johnson	Sparse	0.8469	Yes	$O(nm + n^2 \log n)$	91	13.8203
Dijkstra	Sparse	0.069	No	$O(m + n \log n)$	31	0.6406
Bellman–Ford	Sparse	0.0635	Yes	$O(nm)$	32	0.6328
Floyd–Warshall	Sparse	5.5961	Yes	$O(n^3)$	1141	23.2812
Johnson	Sparse	3.8151	Yes	$O(nm + n^2 \log n)$	861	52.9531
Dijkstra	Sparse	0.1072	No	$O(m + n \log n)$	57	1.0156
Bellman–Ford	Sparse	0.1036	Yes	$O(nm)$	92	0.875
Floyd–Warshall	Sparse	30.935	Yes	$O(n^3)$	4549	73.3203
Johnson	Sparse	12.7996	Yes	$O(nm + n^2 \log n)$	2647	121.5781

Table 1: Comparison of shortest-path algorithms on a Sparse Matrix (n=10)

5 Answers to the Assignment Questions

5.1 Key Differences Between Single-Source and All-Pairs Shortest-Path Algorithms

- **Scope:** Single-source computes distances from one starting vertex to all others; all-pairs computes distances for every pair of vertices.
- **Algorithms:** Single-source uses Dijkstra (non-negative edges) or Bellman–Ford (handles negatives); all-pairs uses Floyd–Warshall (dense/small graphs) or Johnson (sparse, combines Dijkstra & Bellman–Ford).
- **Output:** Single-source returns a vector of length n; all-pairs returns an $n \times n$ distance matrix.
- **Performance Goal:** Single-source minimizes per-source computation; all-pairs either runs a single-source algorithm n times or uses a dedicated $O(n^3)$ approach.
- **Use Cases:** Single-source for routing from one node; all-pairs for global shortest-path queries.

5.2 Why Dijkstra Fails with Negative Edge Weights

- Dijkstra assumes that once a vertex is extracted from the priority queue, its distance is finalized.
- Negative edges may later provide a shorter path to that vertex, breaking this assumption.
- Example: if `source → A = 2`, `A → B = -5`, and `source → B = 1`, Dijkstra may prematurely finalize B and miss the shorter path through A.
- Result: distances can be underestimated; correctness is no longer guaranteed. Use Bellman–Ford when negative edges exist.

5.3 How Johnson Combines Bellman–Ford and Dijkstra

- **Reweighting:** Run Bellman–Ford once on an augmented graph to compute potentials $h(v)$ so that new edge weights $w'(u, v) = w(u, v) + h(u) - h(v)$ are nonnegative.
- **All-Pairs Dijkstra:** Run Dijkstra from every vertex on the reweighted graph.
- **Distance Restoration:** Original distances are recovered via $d(s, t) = d'(s, t) - h(s) + h(t)$.
- **Efficiency:** On sparse graphs, n Dijkstra runs (each $O(m + n \log n)$) plus a single Bellman–Ford $O(nm)$ step can be faster than Floyd–Warshall $O(n^3)$.

5.4 Complexity Analysis of Algorithms

- Dijkstra: Time $O(m + n \log n)$, Memory $O(n + m)$, nonnegative edges only.
- Bellman–Ford: Time $O(nm)$, Memory $O(n + m)$, detects negative cycles.
- Floyd–Warshall: Time $O(n^3)$, Memory $O(n^2)$, simple, good for dense graphs and small n.
- Johnson: Time $O(nm + n^2 \log n)$, Memory $O(n + m + n^2)$, best for sparse graphs needing all-pairs distances.

5.5 Algorithm Suitability Per Graph Type

Based on experimental time analysis:

- **Sparse graphs:** Single-source → "Dijkstra or Bellman–Ford (both perform similarly), Bellman slightly better"; All-pairs → Johnson (2.4× faster than Floyd–Warshall)
- **Dense graphs:** Single-source → Dijkstra (3.8× faster than Bellman–Ford); All-pairs → Johnson (1.3× faster than Floyd–Warshall)
- **Mixed/negative edges:** Single-source → Bellman–Ford (required for safety); All-pairs → Johnson (6.1× faster than Floyd–Warshall)
- **Memory constraints:** Adjacency-list algorithms (Dijkstra/Bellman–Ford) use less memory than matrix-based all-pairs algorithms.

6 Conclusion

This report provides a structured analysis of four shortest-path algorithms. The Python implementation automates experimentation and CSV export, enabling deeper comparison of runtime, relaxations, and memory. Johnson proves best for sparse graphs; Floyd–Warshall suits dense ones; Bellman–Ford is essential when negative edges exist.